

UML, VSM & Process Mining

Drei Perspektiven auf denselben Prozess –
Struktur, Wertfluss und Datenrealität als Dreiklang.

Lehrskript – Tag 2 von 2

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag (8 Unterrichtseinheiten)

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

An Tag 1 haben Sie gelernt, Prozesse mit BPMN und EPC zu *modellieren*. Heute geht es darum, denselben Prozess aus *drei verschiedenen Perspektiven* zu betrachten – als Struktur (UML), als Wertfluss (VSM) und als Datenrealität (Process Mining). Jede der drei Methoden beantwortet eine andere Frage; im Zusammenspiel werden sie zum Steuerungsinstrument.

L1 – Wissen (Reproduktion und Einordnung)

- UML praxisnah einordnen: Use Case Diagram (Wer will was?), Activity Diagram (Wie läuft es ab?).
- VSM-Grundbegriffe kennen: Wertstrom, Bearbeitungszeit (BT), Wartezeit (WT), Durchlaufzeit, WIP, Engpass.
- Wertschöpfung unterscheiden: Value-Adding, Non-Value-Adding, notwendig aber nicht wertschöpfend.
- Process Mining einordnen: Event Log, Discovery, Conformance, Enhancement.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Aus einer Prozessbeschreibung ein UML-Use-Case- oder Activity-Diagramm ableiten.
- Eine VSM-Skizze für einen End-to-End-Prozess erstellen, BT/WT schätzen, Engpass identifizieren.
- Aus einem Mini-Event-Log Prozessvarianten erkennen und Hypothesen für Abweichungen formulieren.
- Die richtige Methode für eine konkrete Fragestellung auswählen – UML, VSM oder Process Mining.

L3 – Management (Entscheidung und Verantwortung)

- Die drei Perspektiven (Struktur, Fluss, Daten) als komplementäres Steuerungssystem nutzen.
- Zielkonflikte (Speed vs. Quality vs. Compliance vs. Kosten) bewusst entscheiden und Trade-offs transparent machen.
- Datenminimum für ein belastbares Operating Model definieren – auch bei unvollständigen Daten.
- Entscheidungen unter Unsicherheit treffen und mit Frühwarnsignalen absichern.

Roter Faden

Modelle sind *Hypothesen*, Daten sind *Realität*, Wertstromsicht ist *Wirkung*. Wer nur modelliert, sieht nicht die Varianten. Wer nur Daten anschaut, versteht nicht die Absicht. Wer nur den Wertstrom betrachtet, übersieht die Verantwortlichkeiten. Der Senior-Skill ist, alle drei Perspektiven *zusammenzuführen*.

1. Einstieg: Drei Perspektiven, eine Realität

Stellen Sie sich vor, Sie sollen einen Auftrag bewerten: “Wir wollen unseren Onboarding-Prozess verbessern.” Sie haben drei mögliche erste Schritte. Erstens: ein Modell zeichnen (BPMN, EPC, UML). Zweitens: den Wertstrom aufnehmen – wie lange dauert was, wo staut es sich? Drittens: in die Daten schauen – was passiert in den Systemen wirklich, in welchen Reihenfolgen?

Jede dieser drei Perspektiven liefert eine andere Wahrheit. Das Modell zeigt, wie es *gedacht* ist. Der Wertstrom zeigt, wo *Zeit und Geld verloren* gehen. Die Daten zeigen, wie es *tatsächlich abläuft*. Und jede dieser drei Wahrheiten kann von den anderen abweichen – oft dramatisch. (vgl. Dumas et al., 2018, Kap. 1)

1.1 Warum eine Sicht nicht reicht

In der Praxis arbeiten Teams oft nur mit *einer* Methode – meist der, die sie zuerst gelernt haben. Das hat berechenbare Folgen:

- Wer nur *modelliert*, beschreibt einen Soll-Prozess, der mit der Realität immer weniger zu tun hat, je älter das Modell wird.
- Wer nur den *Wertstrom* betrachtet, sieht zwar, wo es staut – aber nicht, wer eigentlich entscheidet und wer welche Rolle hat.
- Wer nur *Daten* analysiert, kennt jedes Detail des Ist-Zustands – versteht aber nicht, ob die Abweichungen vom Soll absichtliche Verbesserungen oder unbenannte Risiken sind.

Der Dreiklang löst dieses Problem nicht durch eine “vierte, perfekte” Methode. Er löst es, indem jede der drei Sichten genau das beiträgt, was sie kann – und durch die Kombination Lücken sichtbar werden, die jede einzelne Sicht hätte.

1.2 Was Sie heute können werden

Am Ende des Tages werden Sie in der Lage sein, einen konkreten Geschäftsprozess in drei Perspektiven zu beschreiben: als UML Use Case (Wer will was?), als UML Activity (Wie läuft es ab?), als VSM (Wo geht Zeit verloren?) und als Event-Log-Analyse (Was passiert wirklich?). Wichtiger noch: Sie werden wissen, wann welche Methode lohnt – und wann es Verschwendung wäre, alle drei zu erstellen.

Eselsbrücke: S-F-D

S-F-D – die drei Perspektiven, von links nach rechts:

Struktur – wer macht was, mit wem? (UML)

Fluss – wo geht Zeit/Wert verloren? (VSM)

Daten – was passiert tatsächlich, in welchen Varianten? (Process Mining)

Wer alle drei beherrscht, sieht mehr als die Summe der Teile.

2. UML – Notation für Struktur und Verantwortung

UML (*Unified Modeling Language*) ist mit 14 Diagrammtypen die am breitesten aufgestellte Modellierungssprache überhaupt. In der Geschäftsprozess-Praxis brauchen wir davon meist nur zwei: das *Use Case Diagram* (für Scope und Akteure) und

das *Activity Diagram* (für Abläufe). Wer diese zwei beherrscht, deckt 80 % aller Anwendungsfälle ab. (Fowler, 2003, Kap. 1; OMG, 2017)

2.1 Use Case Diagram – Wer will was?

Ein **Use Case Diagram** beantwortet drei Fragen auf einen Blick: *Wer sind die beteiligten Akteure? Welche Ziele verfolgen sie? Wo verläuft der Scope-Rand des Systems?* Die drei Hauptelemente:

- **Akteur** (Strichmännchen) – eine Rolle, die mit dem System interagiert. Akteure sind *außerhalb* des Systems: Kunde, Sachbearbeiter, externes IT-System, Aufsichtsbehörde.
- **Use Case** (Ellipse) – ein konkretes Ziel, das ein Akteur erreichen will. Benennung: Verb + Objekt (“Bestellung aufgeben”, “Rechnung freigeben”).
- **Systemgrenze** (Rechteck) – markiert den *Scope*: was gehört zum System, was nicht.

Use Case Diagrams sind das beste Werkzeug für die frühe Phase eines Projekts: “Worüber reden wir eigentlich?” Sie zwingen das Team, Scope und Verantwortlichkeiten explizit zu machen, bevor jemand das erste Detail-Diagramm zeichnet.

2.2 Activity Diagram – Wie läuft es ab?

Ein **Activity Diagram** ist im Kern eine Schwester der BPMN, nur mit etwas anderen Symbolen. Es zeigt den Ablauf einer Aktivität (oder eines Use Cases) als Folge von Schritten, Entscheidungen und parallelen Pfaden. Die wichtigsten Elemente:

- **Startknoten** (gefüllter Kreis) und **Endknoten** (Kreis mit Ring).
- **Aktion** (abgerundetes Rechteck) – ein einzelner Schritt.
- **Entscheidungsknoten** (Raute) – analog zum XOR-Gateway in BPMN.
- **Parallelisierung / Synchronisation** (waagerechter Balken) – analog zum AND-Gateway.
- **Swimlane** (vertikaler Streifen) – Verantwortungsbereich, analog zur Lane in BPMN.

UML Activity vs. BPMN

Wer BPMN kennt, kann UML Activity in 15 Minuten lesen – die Logik ist nahezu identisch, nur die Symbole sind anders. Die Unterschiede liegen im Detail (z. B. Datenflüsse, Pins, Interruptible Regions), die im Geschäftsalltag selten gebraucht werden. Faustregel: *BPMN für reine Geschäftsprozesse, UML Activity, wenn der Prozess Teil eines größeren Software-Designs ist.*

2.3 Warum nur diese zwei Diagrammtypen?

UML kennt 14 Diagrammtypen, von Klassendiagramm bis Sequenzdiagramm bis State Machine. In der Geschäftsprozess-Modellierung brauchen Sie davon zwei – und sollten der Versuchung widerstehen, jede neue Frage mit einem neuen Diagrammtyp zu beantworten. Drei Gründe: (Fowler, 2003, S. 13–16)

1. **Akzeptanz.** Mehr als zwei Notationen verwirrt die Zielgruppe – und ein verwirrtes Modell wird nicht genutzt.

2. **Pflegeaufwand.** Jedes Diagramm muss aktuell gehalten werden. Vier Diagramme pro Prozess = vier Stellen, die altern können.
3. **Klarheit der Aussage.** Use Case + Activity beantworten 80 % aller Geschäftsprozess-Fragen. Der Rest gehört in BPMN, VSM oder Process Mining.

2.4 Use Case Diagram konkret – Beispiel CloudWorks Onboarding

Sehen wir uns ein konkretes Beispiel an – das Use Case Diagram, das wir später in der Fallstudie (Sektion 8) ausführlich nutzen. *CloudWorks B2B* ist ein SaaS-Anbieter, der seinen Onboarding-Prozess verbessern möchte.

Akteure (außerhalb der Systemgrenze “Onboarding-System”):

- *Neukunde* – löst den Prozess durch Vertragsabschluss aus, liefert Daten, nimmt das Training an.
- *Sales* – bereitet die Übergabe vor, schreibt das Briefing.
- *Onboarding-Team* – führt die operativen Schritte durch.
- *IT* – konfiguriert Zugänge, importiert Daten.
- *Compliance* – prüft Security und gibt Freigaben.
- *Support* – übernimmt nach Go-Live (Hypercare).

Use Cases (jeweils Verb + Objekt):

1. *Account anlegen* – Sales, Onboarding-Team
2. *Vertrags-/Scope-Check* – Onboarding-Team
3. *Zugänge und Rollen vergeben* – Onboarding-Team, IT
4. *Datenimport durchführen* – IT, Neukunde
5. *Security-/Compliance-Check* – Compliance
6. *Kunden-Training durchführen* – Onboarding-Team, Neukunde
7. *Go-Live-Freigabe erteilen* – Onboarding-Team, Compliance
8. *Hypercare starten* – Support (erste 30 Tage nach Go-Live)

Was uns dieses Diagramm *vor jeder Detailmodellierung* zeigt: Sechs Akteure, acht Use Cases. Es wird klar, dass Compliance ein eigener Akteur ist (nicht “irgendwo bei IT”), dass der Neukunde aktiv mitwirkt (nicht nur Empfänger ist), und dass Support erst nach Go-Live einsetzt. Diese Klarheit am Anfang verhindert die häufigsten Scope-Streitigkeiten später im Projekt.

2.5 Activity Diagram lesen – Symbol-für-Symbol

Ein Activity Diagram für unseren Onboarding-Kernablauf (textuell beschrieben, weil wir keine Grafik einbinden):

Start (gefüllter Kreis im Lane “Sales”)

→ *Sales übergibt Onboarding-Briefing* (Aktion in Lane Sales)

→ **Entscheidung** (Raute): “Kundendaten vollständig?”

Nein-Pfad: → *Daten nachfordern* (Lane Onboarding) → *zurück zu Datenprüfung* (max. 2 Iterationen, dann Eskalation)

Ja-Pfad: → *Zugänge konfigurieren* (Lane IT)

→ **Entscheidung**: “Provisioning abgeschlossen?”

Nein, > 48 h: → *Eskalation an IT-Lead*

Ja: → *Security-/Compliance-Check* (Lane Compliance)

→ **Entscheidung:** “Check bestanden?”

Nein: → *Korrekturen* (Lane Onboarding) → *erneuter Check*

Ja: → *Kunden-Training* (Lane Onboarding) → *Go-Live-Freigabe* → **Ende**

Drei Dinge fallen auf: Jede Entscheidung ist disjunkt (Ja/Nein, kein “vielleicht”), jeder Schleifenpfad hat eine *Obergrenze* (z. B. “max. 2 Iterationen”), und die Lanes machen sichtbar, wer in welchem Schritt verantwortet. Genau diese Disziplin unterscheidet ein lesbares Activity Diagram von einem dekorativen Bild.

2.6 Häufige Fehler in UML-Diagrammen

1. **Akteure als konkrete Personen.** “Frau Meier” statt “Sachbearbeiter”. Folgenfehler: Modell altert mit dem Personalwechsel.
2. **Use Cases als UI-Klicks.** “Auf Button klicken” ist kein Ziel. Use Cases beschreiben *was der Akteur erreichen will*, nicht wie er bedient.
3. **Systemgrenze fehlt oder ist unscharf.** Wenn nicht klar ist, was innerhalb des Systems liegt und was außerhalb, taugt das Diagramm nicht zur Scope-Klärung.
4. **Schleifen ohne Obergrenze.** “Erneut prüfen” → “korrigieren” → “erneut prüfen” kann theoretisch unendlich laufen. Praxis: maximal zwei Iterationen, dann Eskalation.
5. **Mehrere Verantwortliche in einer Lane.** “Sales und Onboarding” – entweder splitten oder als Übergabe modellieren. Ungeklärte Verantwortung ist ein Steuerungsfehler, kein Modellierungs-Komfort.

UML-Disziplin

Die häufigste UML-Falle ist nicht falsche Syntax – es ist *zu viel* UML. Vier Diagramme pro Prozess sind drei zu viel. Wenn ein Use Case Diagram und ein Activity Diagram die Frage nicht beantworten, hilft kein weiterer Diagrammtyp – dann fehlt entweder ein VSM (Zeit/Fluss) oder ein Process Mining (Daten).

3. Value Stream Mapping (VSM) – der Wertfluss

VSM kommt aus der Lean-Produktion und wurde von Mike Rother und John Shook 1999 in dem Klassiker “Learning to See” für die Toyota-Welt dokumentiert. Heute ist VSM eine etablierte Methode auch außerhalb der Produktion – für Verwaltungsprozesse, Software-Entwicklung, Vertriebs- und Onboarding-Prozesse. (Rother & Shook, 1999; Womack & Jones, 2003)

3.1 Die zentrale Frage von VSM

VSM beantwortet eine Frage: *Wo entlang des Wertstroms vom Trigger bis zum Output geht Zeit verloren – und warum?* Es geht *nicht* darum, eine möglichst detailgenaue Prozessbeschreibung zu erzeugen. Es geht darum, die Stellen sichtbar zu machen, an denen *Wartezeiten, Wiederholungen oder unnötige Schleifen* entstehen.

3.2 Die VSM-Elemente

Eine VSM ist im Kern eine horizontale Skizze, die den Prozess von links nach rechts zeigt:

- **Prozessschritt-Box** – ein Schritt im Wertstrom. Pro Box werden zwei Zeiten notiert: **Bearbeitungszeit (BT)** – wie lange dauert die eigentliche Tätigkeit? und **Wartezeit (WT)** – wie lange wartet das Werkstück (oder der Vorgang) zwischen den Schritten?
- **Materialfluss / Informationsfluss** – Pfeile, die zeigen, was zwischen den Schritten weitergegeben wird.
- **Queue / WIP** – ein Dreieck-Symbol für Warteschlangen (“Work in Progress”) zwischen Schritten. Hier sitzen typischerweise die größten Verluste.
- **Push / Pull** – gestrichelte vs. durchgezogene Pfeile, je nachdem, ob ein Schritt “geschoben” (push) oder “gezogen” (pull) wird.

3.3 Bearbeitungszeit, Wartezeit, Durchlaufzeit

Drei Zeiten, die in jeder VSM klar getrennt werden müssen:

- **Bearbeitungszeit (BT)** – die Zeit, in der *tatsächlich am Vorgang gearbeitet wird*. “Bestellung prüfen: 4 Minuten.”
- **Wartezeit (WT)** – die Zeit, in der der Vorgang *in einer Warteschlange liegt*. “Bestellung wartet auf Freigabe: 2 Tage.”
- **Durchlaufzeit (DLZ)** – die Gesamtzeit vom Anfang bis zum Ende des Wertstroms. $DLZ = \text{Summe aller BT} + \text{Summe aller WT}$.

In den meisten Wertströmen ist die WT *ein Vielfaches* der BT. Beispiel Onboarding: “Wir brauchen 6 Wochen.” Wenn man die reine Bearbeitungszeit aufaddiert, kommen oft nur 4–6 Stunden raus. Die anderen 5,5 Wochen sind Wartezeit. Das ist die entscheidende Einsicht: *Optimierung lohnt sich fast immer mehr bei der WT als bei der BT*.

3.4 Wertschöpfung erkennen

Jeder Schritt wird in eine von drei Kategorien einsortiert: (Womack & Jones, 2003, Kap. 2)

1. **Wertschöpfend (Value-Adding, VA)**. Der Kunde würde dafür bezahlen, wenn er es wüsste. “Ware kommissionieren.”
2. **Nicht wertschöpfend, aber notwendig (Necessary Non-Value-Adding, NNVA)**. Notwendig aus regulatorischen oder organisatorischen Gründen, aber kein Wertbeitrag für den Kunden. “Rechnung archivieren.”
3. **Verschwendung (Waste, Muda)**. Weder wertschöpfend noch notwendig. “Bestellung erneut prüfen, weil die erste Prüfung unvollständig dokumentiert war.”

Die sieben Verschwendungsarten (Toyota)

Aus der Lean-Tradition stammen sieben Verschwendungstypen: Überproduktion, Wartezeit, Transport, Überbearbeitung, Bestand, Bewegung, Fehler/Nacharbeit. In Wissensprozessen ergänzt sich oft eine

achte: **ungenutzte Kreativität / Wissen**. Eine gute VSM zeigt, wo welche Art auftritt – und gibt damit Hinweise auf die wirksamste Gegenmaßnahme.

3.5 Engpass-Diagnose mit VSM

Der **Engpass** ist der Schritt im Wertstrom, der die *gesamte Durchlaufzeit* am stärksten begrenzt. Eine VSM macht ihn sichtbar – typischerweise dort, wo die Wartezeit (WT) einen Sprung nach oben macht. Faustregeln zur Engpass-Identifikation:

- Wo wartet WIP am längsten? (höchste WT)
- Wo werden am häufigsten Eskalationen ausgelöst?
- Wo wechselt der Vorgang Rolle, System oder Ort? (Schnittstellen verlieren Zeit)
- Wo werden Vorgänge zurückgewiesen / nachbearbeitet? (Wiederholungs-Schleifen)

Eselsbrücke: **BT-WT-DLZ**

Die drei Zeiten, die jede VSM braucht:

BT = Bearbeitungszeit (was geschieht aktiv)

WT = Wartezeit (wie lange wird gewartet)

DLZ = Durchlaufzeit (Gesamtdauer)

Faustregel: DLZ » Summe BT. Der Großteil der DLZ ist Warten.

3.6 Die sieben Verschwendungsarten im Detail

Toyota hat sieben Verschwendungsarten kodifiziert, die in jedem Prozess auftauchen können. In Wissensprozessen (Büro, IT, Vertrieb) verändern sich die Beispiele, die Logik bleibt:

Typ	Produktion (klassisch)	Wissensprozess (Übersetzung)
Überproduktion	Mehr Teile fertigen als gebraucht	Reports erstellen, die niemand liest; Pflichtfelder, die niemand auswertet
Wartezeit	Maschine wartet auf Material	Vorgang wartet auf Freigabe, Daten, Antwort
Transport	Teile zwischen Standorten bewegen	Daten zwischen Systemen exportieren/importieren
Überbearbeitung	Bauteil feiner schleifen als nötig	Mehrfache Reviews ohne Mehrwert; “Schönschreiben” von Standard-Tickets
Bestand	Lager mit fertigen Teilen vollstellen	Posteingänge / Warteschlangen mit hunderten ungelesenen Tickets
Bewegung	Mitarbeiter laufen zwischen Stationen	System-Wechsel pro Vorgang (CRM → Ticket → ERP → Mail)
Fehler / Nacharbeit	Defekte Teile aussortieren, neu fertigen	Fehlerhaft erfasste Stammdaten, Rückläufer-Tickets, Eskalationen

In modernen Wissensprozessen wird oft eine *achte* Verschwendung ergänzt: **ungenutzte Kreativität / ungenutztes Wissen** – Mitarbeitende, deren Erfahrung systematisch übergangen wird. Das ist die teuerste Verschwendungsart, weil sie die Verbesserung des Prozesses selbst verhindert. (Womack & Jones, 2003, Kap. 2)

3.7 Current State vs. Future State VSM

Eine VSM kommt typischerweise in zwei Versionen vor: **Current State** (wie es jetzt ist) und **Future State** (wie es sein soll). Der Wert entsteht durch die Gegenüberstellung – nicht durch eine der beiden allein. (Rother & Shook, 1999)

- Der *Current State* ist die ehrliche Aufnahme des Ist-Zustands: BT/WT geschätzt oder gemessen, Engpässe identifiziert, Verschwendungen markiert. Ohne diese Ausgangsbasis fehlt jede Veränderungs-Diskussion die Faktengrundlage.
- Der *Future State* ist die kollektive Zielvereinbarung: Wo wollen wir in 6–12 Wochen sein? Welche Verschwendungen sind eliminiert? Welche Wartezeiten halbiert? Wichtig: der Future State ist kein utopisches Wunsch-Diagramm, sondern eine machbare Ziel-Skizze.

Die häufigste Falle: nur einen Current State zeichnen, dann “analysieren”, aber nie einen klaren Future State formulieren. Das Ergebnis: viele Erkenntnisse, keine gemeinsame Richtung.

3.8 VSM in Wissensprozessen – Besonderheiten

VSM stammt aus der Produktion, wird heute aber überwiegend in Wissensprozessen eingesetzt (Verwaltung, IT, Vertrieb, Onboarding). Drei Besonderheiten:

1. **Wertstrom ist unsichtbar.** In der Fabrik sehen Sie das WIP physisch zwischen Maschinen. Im Büro liegt der Vorgang als E-Mail, Ticket oder PDF irgendwo in einer Inbox – visuell unsichtbar, aber genauso real.
2. **Zeiten sind individueller.** Eine Maschine braucht für jedes Teil dieselbe Zeit. Ein Sachbearbeiter braucht für einfache Vorgänge 2 Minuten, für komplizierte 2 Stunden. Varianz ist hoch – Schätzungen brauchen Vertrauensintervalle.
3. **Werte sind kulturell.** “Compliance-Prüfung” ist in der einen Organisation NNVA, in der anderen wertschöpfend (z. B. wenn Compliance ein Marken-USP ist). Wertschöpfungs-Klassifikationen sind nicht universal.

VSM-Pragmatik

Im Wissensprozess sind 80 % einer VSM Schätzung – das ist kein Mangel, sondern unvermeidbar. Die Disziplin: *Schätzungen als solche kennzeichnen, Bandbreiten angeben, prüfen wo Messung lohnt*. Eine geschätzte VSM mit ehrlicher Markierung schlägt eine “präzise” VSM, die im Detail falsch ist.

4. Process Mining – vom Modell zur Datenrealität

Process Mining ist die jüngste der drei Methoden – sie entstand Anfang der 2000er Jahre an der TU Eindhoven (van der Aalst) und ist heute eine etablierte Disziplin mit eigenen Konferenzen, Werkzeugen und einem internationalen Handbuch. (van der Aalst, 2016; van der Aalst & Carmona, 2022)

4.1 Das zentrale Konzept: der Event Log

Process Mining setzt voraus, dass IT-Systeme Spuren hinterlassen – jeder Arbeitsschritt erzeugt einen Eintrag in einer Log-Datenbank. Diese Spuren werden zu einem **Event Log** zusammengeführt. Ein Event Log hat im Minimum drei Felder pro Eintrag:

- **Case ID** – welcher Vorgang? (eine Bestellung, ein Antrag, ein Ticket)
- **Aktivität** – welcher Schritt wurde ausgeführt? (“Antrag prüfen”, “Freigabe erteilen”)
- **Zeitstempel** – wann wurde der Schritt ausgeführt?

Optional, aber wertvoll: **Ressource** (welche Person oder welches System hat die Aktivität ausgeführt?), Datenfelder zum Vorgang (Betrag, Kategorie, Priorität), Kostenstellen.

4.2 Drei Disziplinen des Process Mining

Process Mining wird in drei Disziplinen unterteilt, die unterschiedliche Fragen beantworten: (van der Aalst, 2016, Kap. 1)

1. **Discovery** – “Welcher Prozess läuft hier tatsächlich ab?” Aus den Event Logs wird ein Ist-Prozessmodell rekonstruiert – ohne dass jemand einen Modellierungs-Workshop machen muss. Algorithmen finden die häufigsten Pfade, die Varianten und die Ausreißer.
2. **Conformance Checking** – “Stimmt das Soll mit dem Ist überein?” Ein bestehendes Soll-Modell wird gegen den Event Log gehalten. Abweichungen wer-

den gemessen: Welche Schritte fehlen? Welche Schritte kommen zusätzlich vor? Welche Reihenfolgen werden verletzt?

3. **Enhancement** – “Was lernen wir noch dazu?” Das Modell wird angereichert um Zeitinformationen (wo dauert es wie lange?), um Frequenzen (wie häufig wird ein Pfad genommen?), um Engpässe.

4.3 Was Process Mining nicht ersetzt

Process Mining liefert Datenrealität – aber es ersetzt weder Modellierung noch Wertstromanalyse:

- Es zeigt nicht den *Soll-Prozess*. Die Daten zeigen, was passiert ist – nicht, was passieren sollte.
- Es zeigt nicht die *Absicht*. Warum eine Variante existiert (Fehler? bewusste Anpassung? Ausnahmebehandlung?), beantworten die Logs nicht.
- Es zeigt nicht die *Verantwortlichkeiten*. “Wer ist befugt, diese Entscheidung zu treffen?” steht selten im Log.
- Es ist nur so gut wie die *Datenqualität*. Wenn Schritte nicht protokolliert werden, fehlen sie auch im Discovery-Output.

Process Mining – der Realitätscheck

Process Mining ist die wirksamste Methode, um die Frage zu beantworten: “*Läuft unser Prozess wirklich so, wie wir glauben?*” Die Antwort ist fast immer: *Nein, nicht ganz*. Und genau diese Differenz ist der Goldminen-Moment – weil sie zeigt, wo Aufwand für Verbesserung sich lohnt und wo nicht.

4.4 Voraussetzungen für ein gutes Process-Mining-Projekt

Nicht jeder Prozess eignet sich für Process Mining. Drei Voraussetzungen sollten erfüllt sein: (vgl. van der Aalst & Carmona, 2022, Kap. 1)

1. **Digitale Spuren.** Die Schritte des Prozesses müssen in IT-Systemen protokolliert sein. Reine Papier-Prozesse liefern keine Logs.
2. **Eine eindeutige Case ID.** Ohne Case ID lassen sich die Events nicht zu Vorgängen zusammenfassen. In der Praxis ist die Case ID oft die größte Datenqualitäts-Baustelle: gleicher Vorgang, in drei Systemen drei verschiedene IDs.
3. **Genug Cases.** 50 Cases sind zu wenig. Sinnvoll wird die Analyse meist ab einigen hundert bis tausend Cases – abhängig von der Varianz.

4.5 Ein Mini-Event-Log – konkretes Beispiel

Um zu verstehen, was Discovery und Conformance tatsächlich auswerten, hier ein Mini-Event-Log mit drei Cases aus einem Bestellprozess:

Case ID	Aktivität	Zeitstempel	Ressource
A	Bestellung_eingegangen	2026-05-29 09:00	System
A	Angaben_prüfen	2026-05-29 09:05	Meier
A	Nachfragen	2026-05-29 09:20	Meier
A	Angaben_vollständig	2026-05-29 10:30	Meier
A	Zahlung_prüfen	2026-05-29 10:35	System
A	Versand	2026-05-29 16:00	Schmidt
B	Bestellung_eingegangen	2026-05-29 11:00	System
B	Angaben_prüfen	2026-05-29 11:05	Meier
B	Zahlung_prüfen	2026-05-29 11:06	System
B	Kommissionieren	2026-05-29 14:00	Schmidt
B	Versand	2026-05-29 14:45	Schmidt
C	Bestellung_eingegangen	2026-05-29 08:10	System
C	Angaben_prüfen	2026-05-29 08:20	Meier
C	Zahlung_prüfen	2026-05-29 08:25	System
C	Warten_auf_Nachschub	2026-05-29 08:26	System
C	Kommissionieren	2026-05-31 09:00	Schmidt
C	Versand	2026-05-31 09:30	Schmidt

Was Discovery aus diesen drei Cases erkennen würde:

- *Zwei Prozessvarianten*: Variante 1 (Cases A) mit “Nachfragen”-Schleife, Variante 2 (Cases B, C) ohne Rückfrage.
- Ein *wahrscheinlicher Engpass*: in Case C wartet der Vorgang fast 2 Tage auf Nachschub – dramatisch im Verhältnis zur sonstigen DLZ.
- *Drei Hypothesen für die Abweichungen*: (a) Datenqualität bei Eingang (löst Nachfragen aus), (b) Lagerverfügbarkeit (löst Nachschub-Wartezeit aus), (c) Ressourcen-Engpass bei Schmidt (alle Versand-Aktivitäten konzentriert).
- *Zwei fehlende Datenfelder*, deren Ergänzung Entscheidungen verbessern würde: *Grundcode für Nachfragen* (Schemafehler? Tippfehler? unklar?) und *Lagerstatus zum Bestellzeitpunkt* (war Engpass vorhersehbar?).

Drei Cases reichen natürlich nicht für eine ernsthafte Analyse – in Wirklichkeit braucht es Hunderte bis Tausende. Aber das Prinzip ist sichtbar: Mit *drei Pflichtfeldern* und einer optionalen Ressource lässt sich erstaunlich viel ableiten.

4.6 Discovery-Algorithmen im Überblick

Aus dem Event Log einen Prozess zu rekonstruieren ist nicht-trivial – die Reihenfolge ist klar, aber Verzweigungen, Parallelitäten und Schleifen müssen aus Mustern abgeleitet werden. Drei Algorithmen-Familien dominieren in der Praxis: (van der Aalst, 2016, Kap. 6; van der Aalst & Carmona, 2022, Kap. 4)

- **Alpha-Algorithmus**. Der akademische Klassiker – erkennt Sequenz, Wahl (XOR) und Parallelität (AND) aus den Direkt-Folge-Beziehungen. Schwäche: anfällig für Rauschen (jeder Ausreißer beeinflusst das Modell).
- **Heuristic Miner**. Praxis-tauglicher: filtert seltene Pfade weg, ist robust gegenüber Rauschen, liefert ein Modell, das “die typischen 80 %” der Cases zeigt.

- **Inductive Miner.** Garantiert *wohlgeformte* Modelle (jeder Pfad endet, keine Deadlocks) und ist heute die meistgenutzte Familie in kommerziellen Tools.

Was Sie als Anwender wissen sollten: *Welcher Algorithmus mit welchen Parametern gerade läuft.* Das gleiche Event Log kann mit unterschiedlichen Algorithmen sehr unterschiedliche Modelle produzieren – und keiner davon ist “richtig”, sondern nur unterschiedlich gefiltert.

4.7 Conformance-Metriken

Wenn Sie ein Soll-Modell gegen einen Event Log halten, brauchen Sie Metriken zur Bewertung der Übereinstimmung:

- **Fitness** – wie viele der tatsächlichen Cases lassen sich mit dem Soll-Modell vollständig nachvollziehen? Hohe Fitness = Soll deckt das Ist gut ab.
- **Precision** – wie eng beschreibt das Modell tatsächlich beobachtetes Verhalten – und nicht zusätzlich auch nie beobachtete Pfade? Hohe Precision = Modell ist nicht zu permissiv.
- **Generalization** – wie gut beschreibt das Modell auch *zukünftiges* Verhalten (nicht nur das beobachtete)?
- **Simplicity** – wie kompakt ist das Modell?

Fitness und Precision stehen oft in einem Trade-off zueinander: ein Modell, das jeden Ausreißer abdeckt, hat hohe Fitness, aber niedrige Precision. Ein restriktives Modell hat hohe Precision, aber niedrige Fitness. Die Kunst liegt in der Balance.

4.8 Typische Datenquellen für Event Logs

Aus welchen Systemen kommen die Logs in der Praxis? Sechs typische Quellen:

1. **ERP-Systeme** (SAP, Oracle, Dynamics) – Belege, Vorgänge, Genehmigungsschritte.
2. **CRM-Systeme** (Salesforce, HubSpot, Dynamics) – Leads, Opportunities, Customer-Service-Tickets.
3. **Ticket-Systeme** (Jira, ServiceNow, Zendesk) – IT-Service-Management, Change-Management.
4. **Workflow-Engines** (Camunda, Flowable) – jeder Task-Wechsel ist ein Event.
5. **Application-Logs / Audit Trails** – nutzbar, wenn fachliche Bedeutung mit den technischen Logs verknüpft werden kann.
6. **Datenbank-Tabellen mit Zeitstempel-Feldern** (z. B. `erstellt_am`, `bearbeitet_am`, `abgeschlossen_am`) – liefern indirekt Events durch zeitliche Reihenfolge.

Datenquellen-Realität

In der Praxis kommt ein einziger Event Log selten aus einer einzigen Quelle. Typisch sind 3–5 Quellsysteme, die zusammengeführt werden müssen – mit allen Herausforderungen der Datenkonsistenz: unterschiedliche Case-IDs, abweichende Aktivitätsbezeichnungen, Zeitstempel in verschiedenen Zeitzonen. Genau hier liegt der größte Aufwand jedes Process-Mining-Projekts.

5. Der Methodendreiklang

Bis hier haben wir die drei Methoden einzeln betrachtet. Die Stärke entsteht aber erst durch *die Kombination*.

5.1 Was jede Methode beiträgt – und was sie auslöst

Methoden	Beantwortet	Lässt aus
UML (Use Case + Activity)	Wer? Was? Wie sollte es ablaufen?	Wie dauert es? Was passiert wirklich?
VSM	Wo geht Zeit/Wert verloren?	Wer entscheidet? Welche Varianten gibt es?
Process Mining	Was passiert wirklich, in welchen Varianten?	Wie sollte es sein? Wer trägt Verantwortung?

Die Kombination ist mehr als die Summe der Teile: Wenn UML zeigt, dass eine Rolle entscheiden *sollte*, VSM zeigt, dass an dieser Stelle Wartezeit entsteht, und Process Mining zeigt, dass die Entscheidung in 40 % der Fälle übersprungen wird – dann haben Sie nicht drei Beobachtungen, sondern eine konsistente Diagnose.

5.2 Eine Story, drei Perspektiven

Beispiel “Customer Onboarding” (siehe Sektion 8 für die ausführliche Fallstudie):

- **UML Use Case:** Akteure sind Kunde, Sales, Onboarding-Team, IT, Compliance. Ihre Ziele sind: “Account anlegen”, “Zugänge vergeben”, “Security-Check bestehen”.
- **UML Activity:** Kernablauf mit drei Swimlanes (Sales, Onboarding, IT). Entscheidungspunkte: “Daten vollständig?”, “Security-Check bestanden?”
- **VSM:** 10 Schritte vom Sales-Übergabe bis zum Go-Live. BT zusammen 6 Stunden, DLZ aktuell 6 Wochen. Engpass: IT-Provisioning (Wartezeit dominiert).
- **Process Mining (qualitativ):** 40 % der Cases mit Rücksprüngen wegen fehlender Kundendaten, 25 % mit Wartezeiten > 5 Tagen, 15 % mit Eskalation vor Go-Live.

Erst die Zusammenführung zeigt: Das Problem ist nicht “IT ist langsam”. Das Problem ist, dass die Übergabe vom Sales an Onboarding in 40 % der Fälle unvollständig ist – und die Wartezeit beim IT-Provisioning dadurch entsteht, dass die fehlenden Daten erst nachgefordert werden müssen.

Die goldene Klammer

Modell, Wertstrom und Daten erzählen drei Teile derselben Geschichte. Die *goldene Klammer* um diese drei Teile entsteht, wenn dieselbe Person (oder dasselbe Team) die drei Sichten zusammenführt. Trennt man sie auf – Modellierer hier, Lean-Berater dort, Data Analyst woanders – entstehen drei parallele Berichte, die nie zusammenkommen.

5.3 Drei Mini-Beispiele: wann lohnt welcher Dreiklang?

Damit die Methoden-Kombination nicht abstrakt bleibt, hier drei kontrastierende Mini-Szenarien:

Szenario A – Reklamationsbearbeitung in einem Versandhandel. Die Vermutung: “Bearbeitung dauert zu lange.” Was zeigt was?

- *UML Use Case* – klärt Akteure (Kunde, Kundenservice, Logistik, Buchhaltung) und macht sichtbar, dass Buchhaltung de facto Mit-Entscheider bei Erstattungen ist.
- *VSM* – zeigt: Bearbeitungszeit pro Reklamation 15 Minuten, Durchlaufzeit aber 8 Tage. Größte Wartezeit: “warten auf Erstattungs-Freigabe Buchhaltung”.
- *Process Mining* – entdeckt, dass 30 % der Cases zweimal an die Buchhaltung gehen (erste Freigabe wird nach Rückfrage zurückgezogen).

Diagnose der Kombination: Das eigentliche Problem ist nicht “Buchhaltung ist langsam” – es ist, dass die initialen Freigaben in 30 % der Fälle revidiert werden müssen. Ohne den Mining-Befund hätte man die Kapazität in Buchhaltung erhöht (teuer, falsche Wirkungsrichtung).

Szenario B – Compliance-Check für Vertragsfreigaben. Die Vermutung: “Wir wollen einen neuen Workflow konzipieren, der ihn beschleunigt.”

- *UML Use Case* – klärt, dass es vier Vertragsarten gibt mit unterschiedlichen Akteuren (Standardvertrag, Rahmenvertrag, Sondervertrag, Compliance-Sondercase).
- *UML Activity* – macht den Soll-Workflow je Vertragsart explizit.
- *VSM* – entfällt teilweise: in diesem Greenfield-Projekt gibt es noch keinen verlässlichen Ist-Wertstrom zu vermessen.
- *Process Mining* – entfällt: keine Logs für einen neuen Workflow.

Diagnose: Hier reicht UML aus – VSM und Mining wären verfrüht. *Mehr Methoden hieße hier mehr Aufwand ohne Mehrwert.*

Szenario C – Verlassen einer Plattform durch Lieferanten. Die Vermutung: “Wir verlieren Lieferanten – warum?”

- *UML Use Case* – entfällt, der Prozess ist bekannt.
- *VSM* – könnte Wartezeiten der Lieferanten auf Zahlungen zeigen.
- *Process Mining* – zeigt, dass abwandernde Lieferanten typischerweise drei spezifische Schritte erlebt haben (langwierige Onboarding-Korrektur, mehrfache Provisions-Diskussion, Verspätung von Auszahlungen) – das ist datenbasiert.

Diagnose: Process Mining ist hier die führende Methode, VSM ergänzt. UML wäre Mehraufwand für eine schon klare Struktur.

5.4 Wann der Dreiklang nicht hilft

Der Dreiklang ist kein Allheilmittel. Drei Situationen, in denen seine Anwendung Schaden anrichten kann:

- **Es gibt schlicht keine Daten.** Process Mining ist unmöglich. Dann ist UML + VSM die richtige Wahl, mit ehrlich markierten Schätzungen.
- **Der Prozess ist so neu, dass es noch kein Ist gibt.** VSM und Process Mining wären verfrüht. UML reicht.

- **Das Problem ist eindeutig politisch / kulturell.** “Verantwortung wird nicht übernommen” lässt sich nicht modellieren – hier brauchen Sie Organisationsentwicklung, nicht Methoden.

6. Methodenauswahl: Welche Methode für welche Frage?

In der Praxis ist Zeit knapp. Niemand bekommt das Mandat, jeden Prozess in allen drei Perspektiven aufzunehmen. Die folgende Heuristik hilft beim Priorisieren:

6.1 Vier Fragen – vier Empfehlungen

Wenn die Frage lautet ...	Erste Methode	Sinnvolle Ergänzung
“Was ist überhaupt unser Scope?”	UML Use Case	—
“Wo verliert dieser Prozess Zeit?”	VSM	Process Mining
“Stimmt unser Modell mit der Realität überein?”	Process Mining (Conformance)	UML / BPMN als Soll
“Wie wollen wir den Prozess umbauen?”	UML Activity oder BPMN	VSM für die Ist-Aufnahme

6.2 Drei typische Kombinationsmuster

1. **Modell + Daten** (UML/BPMN + Process Mining): Klassische Konfiguration für Conformance-Projekte. Das Soll ist modelliert, der Ist kommt aus den Logs, die Differenz wird gemessen.
2. **Wertstrom + Daten** (VSM + Process Mining): Stark bei Optimierungsprojekten. Die VSM gibt die Hypothesen, das Process Mining liefert die Zahlen, die VSM wird datenbasiert verfeinert.
3. **Modell + Wertstrom** (UML + VSM): Sinnvoll, wenn (noch) keine guten Daten vorliegen. Die UML klärt Scope und Verantwortung, die VSM zeigt, wo Aufwand für eine Datenerhebung sich lohnen würde.

Methoden-Discipline

Die häufigste Falle ist es, mit *allen* drei Methoden zugleich zu starten. Das Ergebnis sind drei halbgare Artefakte und ein Team, das nicht weiß, welcher Befund welche Konsequenz hat. Besser: *Eine Methode, vollständig, mit klarer Aussage. Dann gezielt ergänzen.*

7. Methodenqualität: Was eine gute Analyse ausmacht

Wie bei BPMN/EPC gelten auch für UML, VSM und Process Mining Qualitätskriterien. Sie sind methodenspezifisch, aber im Geist verwandt mit den GoM (Tag 1, Abschnitt 4).